Javascript : Programmation Asynchrone et « Promise »

Definitions

A- Single Threaded, Synchrounous Execution

JavaScript est un language qui s'execute en une seule file d'execution (Single Thread), par opposition à plusieurs fils d'exécution en paralelle (pralell multi-threaded execution).

i. **Single Thread** : l'environnement d'execution execute une seule commande (insruction) à la fois.
ii. **Synchrounous** : c'est l'execution d'une seule instruction à la fois et dans l'ordre (de leurs apparition dans le script).
iii. **Asynchrounous** : c'est l'execution des instructions différées dans le temps (ex:AJAX call ...) sans perturbé l'execution synchrone.

NB: *under the hood of the browser, may be not.*

Javascript supporte le deux types d'execution Synchrone et Asynchrone.

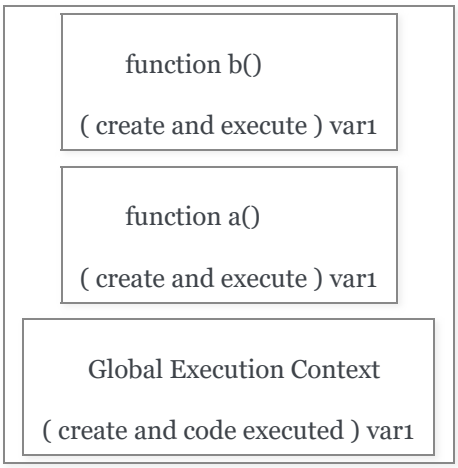
B- Function Invocation and Execution Stack

```
function a(){
    var var1=5;
    b();
 }

 function b(){
    var var1=6;
 }

 var var1=2;
 a();
 console.log(var1);
```

Execution Stack



C- Fonctions Context and Variable Environments

Les variables d'environnement : c'est la place où les varaibles resident et comment cette derinières sont reliées. Chaque contexte d'excution possede son propre environnement.

D- The Scope Chain

Chaque contexte d'execution a une référence a son "outer environment" defini par son "lexical environment".

E- Scope

Scope : (Portée) c'est où une variable est disponible (available) dans le code. (*And it truly the same variable or new copy*)

F- Asynchrounous Callbacks

Asynchrounous : more than one at a time

```
// simulation d'une execution Asynchrone

function asyncStaff(event){
                console.log('asyncStaff starts ....');
                setTimeout(function(...args){

                    console.group('Event');
                    console.log(`%c ${event} : `,
                                "background-color:tomato;color:white;font-size:2rem;")
                    [...args].forEach( console.log );

                    },5000,
                    '{name:"Javascript"}',
                    '{name:"Typescript"}'
                    );
                return  console.log('asyncStaff ends ....');
        }

asyncStaff('GET');

..
```

Callback function : c'est une fonction passée en parametre à une autre fonction (hight order function | fonction d'ordre superieur) pour etre executer plutard. (called back later).

C'est l'ancienne technique utilisée pour ordonner l'execution de deux fonctions asynchrones (ie soit f et g : fonctions asynchrone et on veut excuter g aprés f).

```
function f(callback){
        setTimeout(function(){
                        console.log('doStaff in f');
                        callback();
                    },3000);

        return ({computedValueF:5});
}

function g(){
        setTimeout(function(){
                     console.log('doStaff in g ');
                     },2000);
        return ({computedValueG:10});
}

f(g);  // execute g apres f même si f s'execute aprés 3000ms et g apres 2000ms.
```

Inconvinent: Callback Hell (Callback Doom);

Promise (Promesse ou valeur future)

En Javascript, "Promise" est objet qui retourne une valeur que vous souhaitez recever dans le future, mais pas maintenant. Puisque la valeur sera retourné dans le future, Promise est bien conforme à traiter des

opérations asynchrone.

C'est facile de comprendre Promise par une analogie. Supposons que nous nous donnons une promess d'Apprendre JavaScript le mois prochain: On sait pas si nous parviendrons à cet objectif le mois prochain, alors on a 3 etats de notre promesse:

- Pending (en cours) : on sait pas encore
- Fullfilled (promesse tenue) : On a appris JavaScript
- Rejected (promesse non tenue): On a abondonné l'apprentissage

Promise commense a l'etat Pending (en cours) et se termine soit par Fullfilled (accomplie) ou Rejected (rejettée).

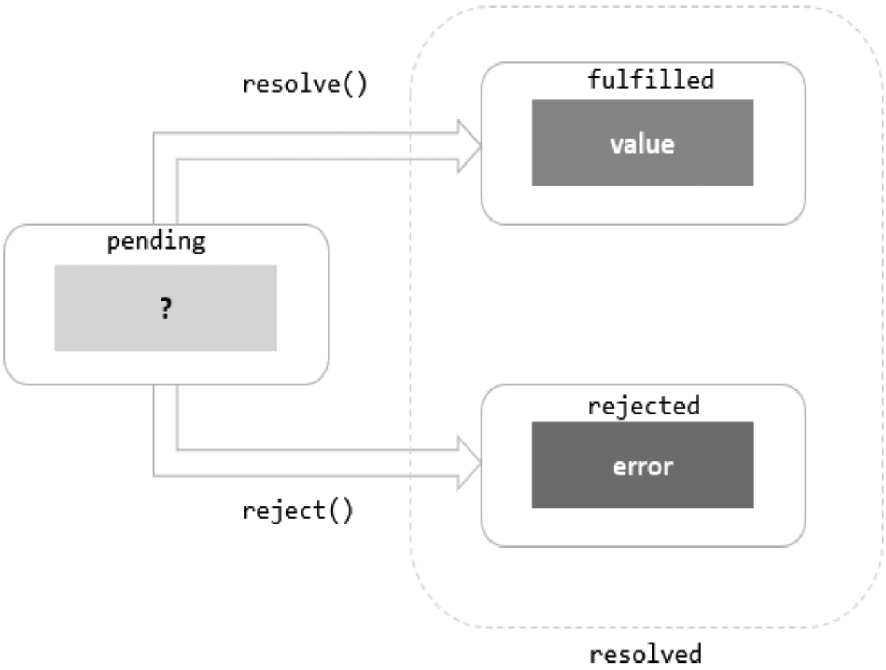
```
let completed=true;

let learnJs = new Promise(
          function( resolve, reject ){
              setTimeout(function(){
                  if(completed){
                      resolve('I have completed learning JS.');
                  } else {
                      reject('I haven\'t completed lerning JS yet.');
                  };
              }, 5 * 1000);
          });


// Consuming the Promise
let futureValue="";
learnJs.then(function(value){
                console.log(value);
                return futureValue=value;
            }).catch(console.error);

console.log(futureValue);
```

ie: *The Promise constructor accepts a function as an argument. This function is called the executor. The executor accepts two functions with the names, by convention, resolve() and reject(). When you call the new Promise(executor), the executor is called automatically. Inside the executor, you manually call the resolve() function if the executor is completed successfully and invoke the reject() function in case of an error occurs.*



Consuming a Promise: then, catch, finally

1- Methode THEN : la methode then enregistre un callback (schedule a callback) pour l'execution une fois « Promise is resolved ». La methode then() prends 2 arguments en parametre (successCallback, failureCallback).

```
// promiseObject.then(onFulfilled,onRejected);

learnJs.then(
  // success
  function(success){
      console.log(success);
  },
  // rejected
  function(reason){
     console.log(reson);
  }
);


// il est possible d'eregistrer que successCallback
learnJS.then(value=>console.log(value));

// ou failureCallback
learnJS.then( undefined, value=>console.log(value) );
```

2- Methode catch : si vous voulez enregistrer juste quand Promise est rejettée. (*Internally, the catch() method invokes the then(undefined, onRejected) method.*)

```
learnJS.catch( raison => console.error(raison))
```

3- Methode finally : parfois on veut excuter du code en depit de resultat de Promise.

```
learnJs.then( success => console.log(success))
       .catch( reason => console.log(reason))
       .finally( () => createApp() );
```

Exemple Pratique de Promise

```
function load(url) {
    return new Promise(function (resolve, reject) {
        const request = new XMLHttpRequest();
        request.onreadystatechange = function ($event) {
            if (this.readyState === 4) {
                if (this.status == 200) {
                    resolve(this.response);
                } else {
                    reject(this.status);
                }
            }
        }
        request.open('GET', url, true);
        request.send();
    })
}

// soit dans HTML : <button id="send">XMLHttpRequest</button>
// et http://localhost:3000/articles est un REST api end-point

const sendButton = document.querySelector('#send');

sendButton.addEventListener('click', function ($event: Event) {
    load('http://localhost:3000/articles')
        .then(
            function (response) {
                console.log(response);
        })
```

```
        .catch(
            function (raison) {
                console.error(raison)
            })
        .finally(function () {
                console.log('Request Completed');
        });
});
```

Async/Await (Built on Promises and Generators)

Async/Await est introduit pour reduire la verbosité du code utilisé par Promise.